

1. Find Maximum and Minimum using Divide and Conquer

Program:

```
#include <stdio.h>

int a[] = {5, 7, 3, 4, 9, 12, 6, 2};

void findMinMax(int left, int right, int *min, int *max)
{
    if (left == right)
    {
        *min = *max = a[left];
        return;
    }

    int mid = (left + right) / 2;
    int min1, max1, min2, max2;

    findMinMax(left, mid, &min1, &max1);
    findMinMax(mid + 1, right, &min2, &max2);

    *min = (min1 < min2) ? min1 : min2;
    *max = (max1 > max2) ? max1 : max2;
}

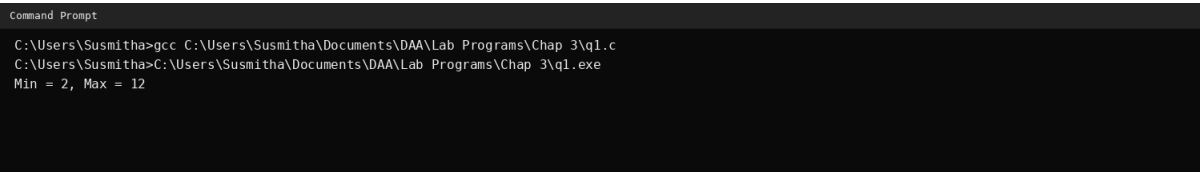
int main()
{
    int min, max;

    findMinMax(0, 7, &min, &max);

    printf("Min = %d, Max = %d\n", min, max);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q1.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q1.exe
Min = 2, Max = 12
```

2. Maximum and Minimum in a Sorted Array

Program:

```
#include <stdio.h>

int a[] = {2, 4, 6, 8, 10, 12, 14, 18};

void findMinMax(int left, int right, int *min, int *max)
{
    if (left == right)
    {
        *min = *max = a[left];
        return;
    }

    int mid = (left + right) / 2;
    int min1, max1, min2, max2;

    findMinMax(left, mid, &min1, &max1);
    findMinMax(mid + 1, right, &min2, &max2);

    *min = (min1 < min2) ? min1 : min2;
    *max = (max1 > max2) ? max1 : max2;
}

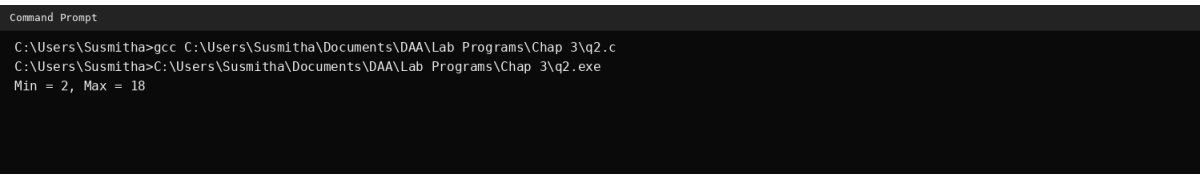
int main()
{
    int min, max;

    findMinMax(0, 7, &min, &max);

    printf("Min = %d, Max = %d\n", min, max);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_3\q2.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_3\q2.exe
Min = 2, Max = 18
```

3. Merge Sort

Program:

```
#include <stdio.h>

void merge(int a[], int left, int mid, int right)
{
    int temp[20];
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (i = left, k = 0; i <= right; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);
        merge(a, left, mid, right);
    }
}

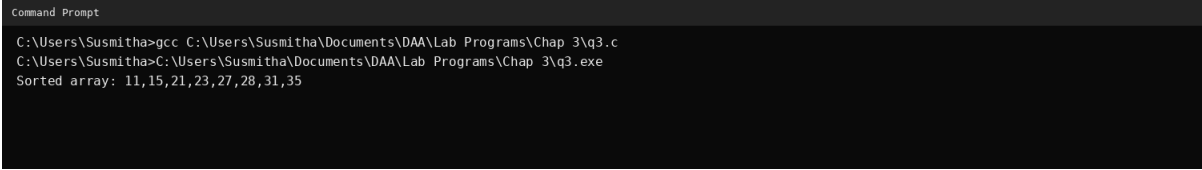
int main()
{
    int a[] = {31, 23, 35, 27, 11, 21, 15, 28};
    int n = 8;

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
```

```
    for (int i = 0; i < n; i++)  
        printf("%d%s", a[i], i == n - 1 ? "\n" : ",");  
  
    return 0;  
}
```

Output:



```
Command Prompt  
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q3.c  
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q3.exe  
Sorted array: 11,15,21,23,27,28,31,35
```

4. Merge Sort with Comparison Count

Program:

```
#include <stdio.h>

long comparisons = 0;

void merge(int a[], int left, int mid, int right)
{
    int temp[20];
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right)
    {
        comparisons++;

        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (i = left, k = 0; i <= right; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);
        merge(a, left, mid, right);
    }
}

int main()
{
    int a[] = {12, 4, 78, 23, 45, 67, 89, 1};
    int n = 8;
```

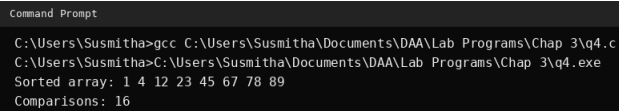
```
mergeSort(a, 0, n - 1);

printf("Sorted array: ");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\nComparisons: %ld\n", comparisons);

return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q4.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q4.exe
Sorted array: 1 4 12 23 45 67 78 89
Comparisons: 16
```

5. Quick Sort using First Element as Pivot

Program:

```
#include <stdio.h>

int a[] = {10, 16, 8, 12, 15, 6, 3, 9, 5};
int n = 9;

int partition(int low, int high)
{
    int pivot = a[low];
    int i = low + 1;
    int j = high;

    while (i <= j)
    {
        while (i <= high && a[i] <= pivot)
            i++;

        while (a[j] > pivot)
            j--;

        if (i < j)
        {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    int temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

void quickSort(int low, int high)
{
    if (low < high)
    {
        int p = partition(low, high);

        printf("After partition: ");
        for (int i = 0; i < n; i++)
            printf("%d%s", a[i], i == n - 1 ? "\n" : ",");

        quickSort(low, p - 1);
    }
}
```

```

        quickSort(p + 1, high);
    }
}

int main()
{
    quickSort(0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d%s", a[i], i == n - 1 ? "\n" : ",");

    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q5.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q5.exe
After partition: 6,5,8,9,3,10,15,12,16
After partition: 3,5,6,9,8,10,15,12,16
After partition: 3,5,6,9,8,10,15,12,16
After partition: 3,5,6,8,9,10,15,12,16
After partition: 3,5,6,8,9,10,12,15,16
Sorted array: 3,5,6,8,9,10,12,15,16

```

6. Quick Sort using Middle Element as Pivot

Program:

```
#include <stdio.h>

void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

void quickSort(int a[], int low, int high)
{
    int i = low;
    int j = high;
    int pivot = a[(low + high) / 2];

    while (i <= j)
    {
        while (a[i] < pivot)
            i++;

        while (a[j] > pivot)
            j--;

        if (i <= j)
        {
            swap(&a[i], &a[j]);
            i++;
            j--;
        }
    }

    printf("Partitioned: ");
    for (int k = low; k <= high; k++)
        printf("%d%s", a[k], k == high ? "\n" : ",");

    if (low < j)
        quickSort(a, low, j);

    if (i < high)
        quickSort(a, i, high);
}

int main()
{
    int a[] = {19, 72, 35, 46, 58, 91, 22, 31};
```

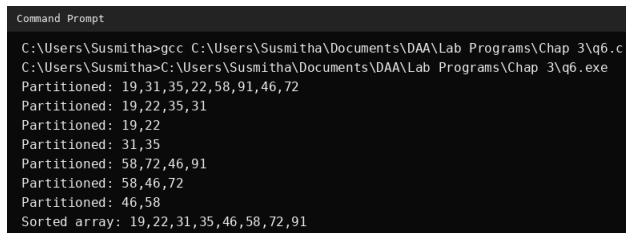
```
int n = 8;

quickSort(a, 0, n - 1);

printf("Sorted array: ");
for (int i = 0; i < n; i++)
    printf("%d%s", a[i], i == n - 1 ? "\n" : ",");

return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q6.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q6.exe
Partitioned: 19,31,35,22,58,91,46,72
Partitioned: 19,22,35,31
Partitioned: 19,22
Partitioned: 31,35
Partitioned: 58,72,46,91
Partitioned: 58,46,72
Partitioned: 46,58
Sorted array: 19,22,31,35,46,58,72,91
```

7. Binary Search with Comparison Count

Program:

```
#include <stdio.h>

int main()
{
    int a[] = {5, 10, 15, 20, 25, 30, 35, 40, 45};
    int n = 9;
    int key = 20;
    int low = 0, high = n - 1;
    int comparisons = 0;

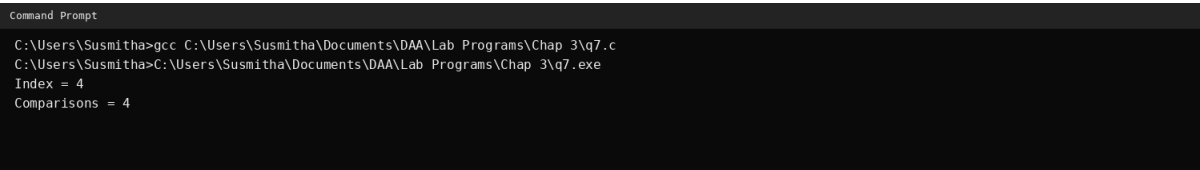
    while (low <= high)
    {
        int mid = (low + high) / 2;
        comparisons++;

        if (a[mid] == key)
        {
            printf("Index = %d\n", mid + 1);
            printf("Comparisons = %d\n", comparisons);
            return 0;
        }

        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    printf("Element not found\n");
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_3\q7.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_3\q7.exe
Index = 4
Comparisons = 4
```

8. Binary Search with Mid-point Calculations

Program:

```
#include <stdio.h>

int main()
{
    int a[] = {3, 9, 14, 19, 25, 31, 42, 47, 53};
    int n = 9;
    int key = 31;
    int low = 0, high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

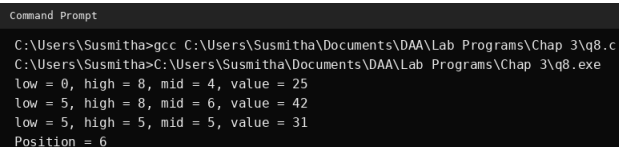
        printf("low = %d, high = %d, mid = %d, value = %d\n",
            low, high, mid, a[mid]);

        if (a[mid] == key)
        {
            printf("Position = %d\n", mid + 1);
            return 0;
        }

        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    printf("Element not found\n");
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q8.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q8.exe
low = 0, high = 8, mid = 4, value = 25
low = 5, high = 8, mid = 6, value = 42
low = 5, high = 5, mid = 5, value = 31
Position = 6
```

9. K Closest Points to Origin

Program:

```
#include <stdio.h>

typedef struct
{
    int x;
    int y;
    int distance;
} Point;

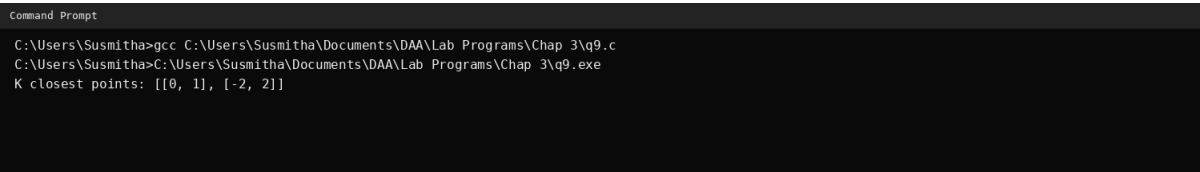
int main()
{
    Point p[] = {{1, 3, 10}, {-2, 2, 8}, {5, 8, 89}, {0, 1, 1}};
    int n = 4;
    int k = 2;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (p[j].distance < p[i].distance)
            {
                Point temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    printf("K closest points: [");
    for (int i = 0; i < k; i++)
        printf("[%d, %d] %s", p[i].x, p[i].y, i == k - 1 ? "" : ", ");
    printf("]\n");

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q9.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q9.exe
K closest points: [[0, 1], [-2, 2]]
```

10. 4Sum – Count Tuples with Zero Sum

Program:

```
#include <stdio.h>

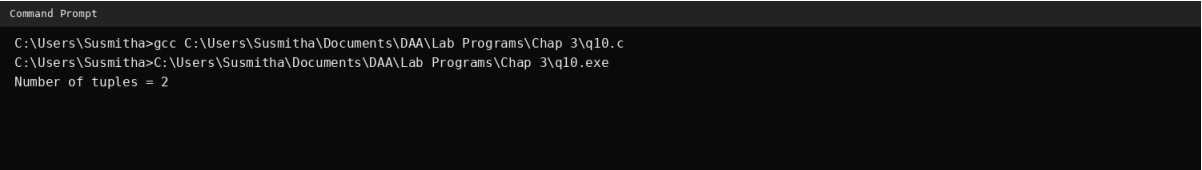
int main()
{
    int A[] = {1, 2};
    int B[] = {-2, -1};
    int C[] = {-1, 2};
    int D[] = {0, 2};
    int n = 2;
    int count = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                for (int l = 0; l < n; l++)
                    if (A[i] + B[j] + C[k] + D[l] == 0)
                        count++;

    printf("Number of tuples = %d\n", count);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q10.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q10.exe
Number of tuples = 2
```

11. Median of Medians – k-th Smallest Element

Program:

```
#include <stdio.h>

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[i])
            {
                int t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
}

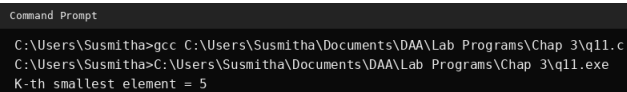
int main()
{
    int a[] = {12, 3, 5, 7, 19};
    int k = 2;
    int n = 5;

    sort(a, n);

    printf("K-th smallest element = %d\n", a[k - 1]);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q11.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q11.exe
K-th smallest element = 5
```

12. Median of Medians Function

Program:

```
#include <stdio.h>

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[i])
            {
                int t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
}

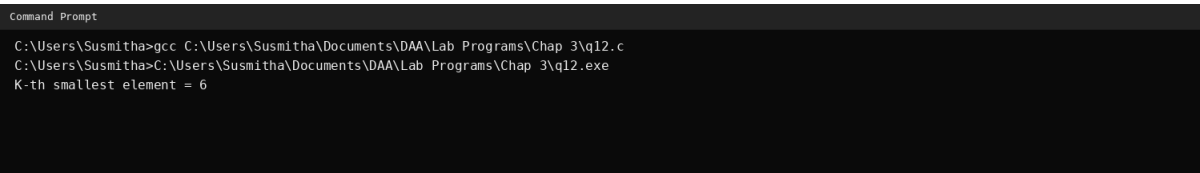
int median_of_medians(int arr[], int n, int k)
{
    sort(arr, n);
    return arr[k - 1];
}

int main()
{
    int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int n = 10;
    int k = 6;

    printf("K-th smallest element = %d\n", median_of_medians(a, n, k));

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q12.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q12.exe
K-th smallest element = 6
```

13. Meet in the Middle – Closest Subset Sum

Program:

```
#include <stdio.h>
#include <stdlib.h>

int a[] = {45, 34, 4, 12, 5, 2};
int n = 6;
int target = 42;
int bestSum = 0;
int bestDiff = 1000000;

void generate(int index, int sum)
{
    if (index == n)
    {
        int diff = abs(target - sum);

        if (diff < bestDiff)
        {
            bestDiff = diff;
            bestSum = sum;
        }
        return;
    }

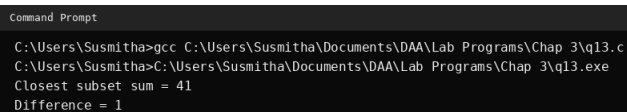
    generate(index + 1, sum);
    generate(index + 1, sum + a[index]);
}

int main()
{
    generate(0, 0);

    printf("Closest subset sum = %d\n", bestSum);
    printf("Difference = %d\n", bestDiff);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q13.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q13.exe
Closest subset sum = 41
Difference = 1
```

14. Meet in the Middle – Exact Subset Sum

Program:

```
#include <stdio.h>

int a[] = {1, 3, 9, 2, 7, 12};
int n = 6;
int target = 15;
int found = 0;

void search(int index, int sum)
{
    if (sum == target)
    {
        found = 1;
        return;
    }

    if (index == n || sum > target)
        return;

    search(index + 1, sum);

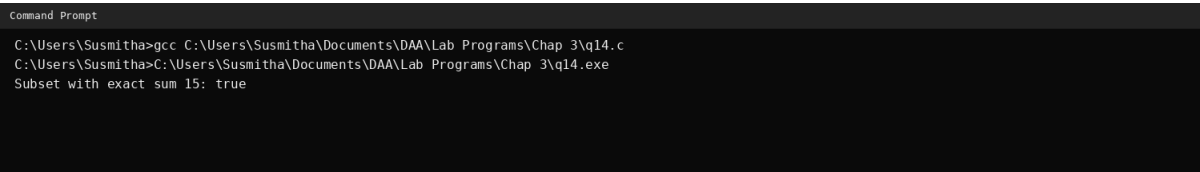
    if (!found)
        search(index + 1, sum + a[index]);
}

int main()
{
    search(0, 0);

    printf("Subset with exact sum %d: %s\n",
        target, found ? "true" : "false");

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q14.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q14.exe
Subset with exact sum 15: true
```

15. Strassen Matrix Multiplication

Program:

```
#include <stdio.h>

void strassen(int A[2][2], int B[2][2], int C[2][2])
{
    int M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
    int M2 = (A[1][0] + A[1][1]) * B[0][0];
    int M3 = A[0][0] * (B[0][1] - B[1][1]);
    int M4 = A[1][1] * (B[1][0] - B[0][0]);
    int M5 = (A[0][0] + A[0][1]) * B[1][1];
    int M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
    int M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);

    C[0][0] = M1 + M4 - M5 + M7;
    C[0][1] = M3 + M5;
    C[1][0] = M2 + M4;
    C[1][1] = M1 - M2 + M3 + M6;
}

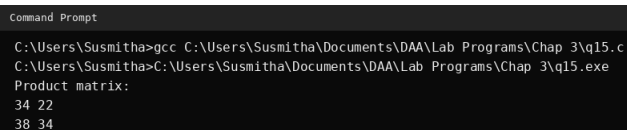
int main()
{
    int A[2][2] = {{1, 7}, {3, 5}};
    int B[2][2] = {{6, 8}, {4, 2}};
    int C[2][2];

    strassen(A, B, C);

    printf("Product matrix:\n");
    printf("%d %d\n", C[0][0], C[0][1]);
    printf("%d %d\n", C[1][0], C[1][1]);

    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q15.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q15.exe
Product matrix:
34 22
38 34
```

16. Karatsuba Multiplication

Program:

```
#include <stdio.h>

long long karatsuba(long long x, long long y)
{
    if (x < 10 || y < 10)
        return x * y;

    long long power = 1;
    long long temp = x;

    while (temp >= 10)
    {
        power *= 10;
        temp /= 10;
    }

    long long a = x / power;
    long long b = x % power;
    long long c = y / power;
    long long d = y % power;

    long long ac = karatsuba(a, c);
    long long bd = karatsuba(b, d);
    long long abcd = karatsuba(a + b, c + d);

    long long adbc = abcd - ac - bd;

    return ac * power * power + adbc * power + bd;
}

int main()
{
    long long x = 1234;
    long long y = 5678;

    printf("%lld x %lld = %lld\n", x, y, karatsuba(x, y));

    return 0;
}
```

Output:

Command Prompt

```
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q16.c  
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 3\q16.exe  
1234 x 5678 = 7006652
```